

Multitasking in Python

Program

A **program** is a set of static instructions written in any programming language to perform a specific task.

Example: A simple program that adds two numbers:

```
a = 10
b = 5
print("Sum =", a + b)
```

Process

A **process** is a program in **execution**. It has its own memory space, code, data, and system resources. Every process runs independently.

To check running processes in Python:

```
import os
print("Current Process ID:", os.getpid())
```

PID – Process ID

Each process is given a **unique ID** called a **PID (Process ID)** by the operating system to identify it.

Use `os.getpid()` to get PID:

```
import os
print("PID:", os.getpid())
```

Thread

A **thread** is the smallest unit of execution within a process. A process can have **multiple threads** sharing the same memory space.

Difference:

- **Process** = Runs independently with its own memory.
- **Thread** = Runs within a process and shares the same memory.

TID – Thread ID

Each thread inside a process has a unique **Thread ID**.

Get Thread ID using `threading.get_ident()`:

```
import threading

def task():
    print("Thread ID:", threading.get_ident())

t1 = threading.Thread(target=task)
t1.start()
t1.join()
```

Address Space of a Process

The **address space** refers to the memory allocated to a process. All threads in a process share:

- Code
- Data
- File descriptors
- Heap & stack (different stack for each thread)

Threads share the same address space. Processes do **not**.

Multitasking

Multitasking is the ability of the CPU to execute **multiple tasks** simultaneously or in quick succession.

Types:

- **Process-based multitasking:** Running multiple applications.
- **Thread-based multitasking:** Multiple threads in a single application.

Multithreading

Multithreading is a technique where **multiple threads** run concurrently within a single process.

Python Multithreading Example:

```
python

import threading

def fun():
    print("Hello from", threading.current_thread().name)

t1 = threading.Thread(target=fun)
t2 = threading.Thread(target=fun)

t1.start()
```

```
t2.start()
```

```
t1.join()
```

```
t2.join()
```

Multicore Programming

Multicore programming means using **multiple cores of a CPU** to execute code in **parallel**, increasing performance.

Python's **multiprocessing** module is used for this.

Example: Using multiple processes:

```
from multiprocessing import Process
import os

def fun():
    print("Running process with PID:", os.getpid())

if __name__ == "__main__":
    for _ in range(4):
        p = Process(target=fun)
        p.start()
```

multiprocessing.Pool – Parallel Task Execution Made Easy

Pool allows you to **manage a pool of worker processes** to run tasks in parallel. It automatically handles process creation and load distribution.

Example: Square numbers using Pool

```
def fun(no):
    print("PID is : ",os.getpid())
    sum = 0
    for i in range(1,no):
        sum = sum + (i*i*i)
    return sum
```

```
def main():
    start_time = time.time()
```

```
data = [1000000,2000000,3000000,4000000,5000000,6000000,7000000,8000000,9000000,10000000]
```

```
result = []

p = multiprocessing.Pool()
result = p.map(fun,data)
p.close()
p.join()
print(result)

end_time = time.time()
print("Execution time is : ",end_time - start_time)

if __name__ == "__main__":
    main()
```

